SOFTWARE ENGINEERING PROJECT REPORT

REQUIREMENTS SPECIFICATION, UML DESIGN & PROTOTYPE IMPLEMENTATION

SYSTEM NAME	DOCUMENT STANDARD	IMPLEMENTATION	TOTAL MARKS
Hotel Management System	**IEEE-830 Standard**	**Python / Tkinter GUI**	**20 Marks**

TASK 1: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

1. System Overview & Objectives

The **Hotel Management System (HMS)** is a comprehensive desktop platform designed to streamline operations for hotel staff and administration. The system automates key hospitality functions including room inventory tracking, guest check-ins, multi-night stay booking, billing receipt generation, dynamic real-time inventory search, and account management while maintaining persistent local file storage.

Intended User Roles:

- **Hotel Staff:** Performs daily operational activities including processing customer room bookings, handling check-outs, searching guest records, and viewing/printing receipts.
- **Hotel Administrator:** Possesses full operational rights along with exclusive security access to manage employee accounts (add/delete staff) and manipulate inventory (add, edit attributes, or delete rooms).

2. Functional Requirements (10 Identifiers)

Req ID	Function	Detailed Requirement Description
FR-001	User Authentication	The system shall authenticate users via username and password against credentials in `staff.txt`, assigning operational access based on `admin` or `staff` roles.
FR-002	Staff Management	The system shall allow administrators to add new staff/admin accounts and delete existing accounts (excluding the active user session).
FR-003	Room Creation	The system shall allow administrators to create room records containing Room Number, Price per Night, Capacity, and Room Type (Standard/Suite).
FR-004	Room Editing	The system shall enable administrators to modify the night pricing and occupant capacity of existing rooms.
FR-005	Room Deletion	The system shall allow administrators to remove unreserved (vacant) rooms from the global inventory list.
FR-006	Room Booking	The system shall allow staff to reserve vacant rooms by entering validated guest contact information (Name, Phone, Email) and duration of stay in nights.
FR-007	Guest Check-Out	The system shall process guest check-outs by transitioning occupied rooms back to vacant status and clearing attached guest attributes.
FR-008	Receipt Generation	The system shall generate itemized financial receipts displaying stay metrics and export printable text files (`Receipt_Room_X.txt`).
FR-009	Dynamic Search	The system shall dynamically filter active room records upon key release across parameters including Room Number, Guest Name, Phone, Email, Type, or Capacity.

Req ID	Function	Detailed Requirement Description
FR-010	Data Persistence	The system shall maintain persistence by auto-syncing all room and account states to local data files (`data.txt` and `staff.txt`) upon every operational state change.

3. Non-Functional Requirements (5 Objective Criteria)

NFR ID	Category	System Goal	Measurable Verification Criteria
NFR-001	Performance	Dynamic Real-Time Search Response	List updates must render within **≤ 200 ms** upon key release in the search bar.
NFR-002	Security	Role-Based Access Control	Admin controls (Room modifications, Staff management) must be completely disabled and hidden for users logged in under the `staff` role.
NFR-003	Data Integrity	Input Field Validation	Phone numbers must strictly validate numeric digits (`isdigit()`) and emails must require `@` and `.` before accepting a booking.
NFR-004	Usability	Active Session Monitoring	The GUI status bar must update active elapsed session time continuously at precisely **1.0 second intervals**.
NFR-005	Availability	Fault-Tolerant State Recovery	If configuration files are missing at launch, the system must create default fallback records (`admin:admin123`) without throwing unhandled exceptions.

TASK 2: SYSTEM DESIGN REPORT (UML DIAGRAMS)

1. Use-Case Diagram & Tabular Descriptions

```
+-------------------------------------------------------------------------------+ | Hotel Management Sys
```

Tabular Description: UC-01 (Make Room Booking)

Actors: Staff, Admin | **Preconditions:** Selected room exists with status 'Available'.

Main Flow: 1. Actor selects vacant room. 2. Inputs validated Guest Name, Phone Number, and Email. 3. Defines total nights. 4. System updates room status to Occupied, writes data to `data.txt`, and launches receipt GUI.

Alternative Flow: If validation fails (e.g., non-numeric phone), system re-prompts actor until valid input is received.

Tabular Description: UC-02 (Manage Staff Accounts)

Actors: Admin | **Preconditions:** Active session has 'Admin' authority.

Main Flow: 1. Admin chooses action ('add' or 'del'). 2. For addition, enters username, password, and role. 3. System updates active memory dictionary and persists changes directly to `staff.txt`.

Alternative Flow: If attempting to delete active admin account or existing username, an error window prevents execution.

2. Sequence Diagram (Key Scenario: Room Reservation Workflow)

Illustrates the chronological interaction sequence between System Actors, GUI Controller, Domain Objects, and File Storage during a booking operation.

```
Staff / Guest HotelGUI (UI) Room (Object) File System (data.txt) | | | | |--- 1. Click Booking ------>| | |
```

3. Activity Diagram (Check-Out Process Workflow)

Represents the operational decisions and sequential processing states during guest departure.

```
[Start: Select Room Check-Out] | v [Prompt for Room Number] | v /--------------------\ < Is Room Occupied? >
```

4. State Diagram (Room Lifecycle States)

Models the dynamic state transitions of a room entity within the hotel management database.

```
+---------------+ Add Room (FR-003) +---------------+ | [UNCREATED] | ----------------------------------
```

5. Class Diagram

```
+-----------------------------------------------------------------------------------------------+ | Room
```

TASK 3: FUNCTIONAL PROTOTYPE IMPLEMENTATION

1. Requirement Prioritization Matrix (Top 5 Selected)

Rank	Req ID	Requirement Description	Status	Selection Justification
1	FR-001	User Authentication & Access Roles	**Selected (1)**	Essential entry portal for security verification.
2	FR-006	Room Reservation Workflow	**Selected (2)**	Primary business feature of the hotel software.
3	FR-007	Guest Check-Out Process	**Selected (3)**	Core operation to clear rooms and manage availability.
4	FR-008	Receipt Generation & Printing	**Selected (4)**	Required financial output verification component.
5	FR-010	Local Data Storage (File I/O)	**Selected (5)**	Critical storage layer ensuring cross-session persistence.

2. Code Traceability & Implementation Proof

```
# FR-001: Authentication & Role Verification def attempt_login(self, e=None): u, p = u_entry.get().
strip().lower(), p_entry.get().strip() if u in self.staff_accounts and self.staff_accounts[u]["pas
s"] == p: self.current_user, self.user_role = u, self.staff_accounts[u]["role"] w.destroy(); self.r
oot.deiconify(); self.build_gui(); self.refresh_list() # FR-006 & FR-010: Reservation Workflow with
Persistence def make_booking(self): if not (r := self._get_target_room("Booking", "Enter Room Numbe
r to book:")): return if not r.available: return mb.showwarning("Unavailable", f"Room {r.num} is oc
cupied.") # Validated Inputs: Name, Phone (digits), Email (@ and .) r.guest_name, r.guest_phone, r.
guest_email, r.nights, r.available = gn, gp, ge, n, False self.save_data() # Persists to data.txt s
elf.show_receipt(r) # Triggers FR-008
```